

Real-Time Posture Detection System Using ESP32 and TensorFlow Lite Micro

Álvaro Terroso, Joel Silva, José Sousa

Escola Superior de Tecnologia – IPCA, Barcelos, Portugal
{a34573, a26030, a34581}@alunos.ipca.pt

Abstract. We present a real-time embedded system for posture classification using dual inertial measurement units (MPU6050) placed on the lower back and chest of a user. Sensor data is transmitted via UDP over WiFi at 50 Hz from two ESP32 nodes to a central ESP32 device, which performs on-device inference using a quantized 1D Convolutional Neural Network deployed with TensorFlow Lite Micro.

The system classifies posture into three categories (*bad_back*, *bad_front*, *good*) and achieves a Stratified Group K-Fold cross-validated accuracy of 94.3 % (± 11.4 %). The deployed model occupies 18.9 KB of flash memory and executes inference in approximately 20 ms after an initial 2-second buffering phase.

This work demonstrates that compact quantized neural networks can achieve highly competitive performance for human posture monitoring on resource-constrained microcontrollers, enabling fully on-device inference without cloud dependency.

1 Introduction

Work-related musculoskeletal disorders represent one of the most prevalent occupational health burdens globally. Among office and sedentary workers, studies consistently report this disorders prevalence rates above 80 % [6], with lower back pain (52–66 %) and neck pain (48–60 %) as the most frequently affected regions [6,1]. In the United States alone, the private sector recorded over 937,620 musculoskeletal disorder cases requiring days away from work or job transfer in 2023–2024 [10]. Prolonged sedentary posture and suboptimal trunk orientation are established biomechanical risk factors [4], making continuous, objective posture monitoring a clinically motivated intervention target.

Inertial measurement units (IMUs) combining accelerometers and gyroscopes have emerged as the dominant sensing modality for wearable posture and activity monitoring, owing to their low cost, small form factor, and absence of line-of-sight constraints [7,5]. Multi-node IMU configurations placed at anatomically meaningful locations (e.g., lumbar spine and sternum) provide richer kinematic context than single-sensor systems, enabling finer discrimination between trunk flexion patterns [7]. However, fusing independently clocked sensor streams introduces temporal synchronization challenges that are frequently underreported in the literature.

On the machine learning side, one-dimensional convolutional neural networks (1D CNNs) have demonstrated strong performance on raw IMU time series, learning task-discriminative temporal features directly from windowed signals without manual feature engineering [9,2]. Fixed-length sliding windows (typically of 1/2 seconds at 50/100 Hz) are the standard segmentation strategy [8]. For deployment on resource-constrained embedded hardware, compact CNN architectures combined with TensorFlow Lite Micro (TFLite Micro) and post-training integer quantization [3] enable on-device inference within kilobytes of flash and hundreds of kilobytes of RAM [9], eliminating cloud dependency, reducing latency, and preserving data privacy.

Existing embedded posture systems either rely on classical machine learning with hand-crafted features [5] or deploy deep models on more powerful edge processors (Raspberry Pi, NVIDIA Jetson) rather than bare-metal microcontrollers [9]. Deployment directly on an ESP32-class microcontroller with 520 KB SRAM and no operating system remains a more constrained and less studied target. Furthermore, evaluation methodology in small-dataset posture studies frequently suffers from session-level data leakage when overlapping windows are assigned to folds at random, a bias that inflates reported accuracy and is rarely addressed explicitly.

This work makes the following contributions:

- A complete end-to-end pipeline for dual-node IMU posture classification, from raw data collection on ESP32 nodes through model training and on-device TFLite Micro inference, fully described and reproducible.
- A nearest-neighbor temporal merge strategy that synchronizes two independently clocked ESP32 sensor nodes at the session level, correctly handling per-device clock resets.
- A methodologically rigorous evaluation using Stratified Group K-Fold cross-validation ($k=5$) that prevents window-level leakage from 50%-overlapping windows, being this a source of bias identified and corrected during development, including an explicit validation of zero session overlap per fold.
- An ablation study of data augmentation strategies (Gaussian noise, time shifting) and architecture regularization (Batch Normalization, L2, Dropout), demonstrating that Gaussian noise augmentation alone yields the best generalization gain (+15.7 pp) while time shifting degrades performance for quasi-static posture tasks.
- Identification and correction of a validation leakage bug introduced by using the held-out test fold as the early stopping validation set, with quantification of its effect ($0.998 \rightarrow 0.943$).
- A fully deployed int8-quantized CNN (18.9 KB flash, ~ 20 ms inference) on an ESP32-WROOM-32, with documented firmware architecture including tensor arena sizing, op resolver configuration, and the complete normalization–quantization pipeline.

2 System Architecture

The system follows a distributed edge inference architecture composed of three autonomous ESP32 nodes communicating over a local WiFi network.

Figure 1 illustrates the data flow. Two sensor nodes, one placed on the lumbar region and one on the sternum, continuously sample the MPU6050 at 50 Hz and transmit raw IMU readings as UDP datagrams to a central inference node. The central node maintains a sliding window of the 100 most recent synchronized samples and invokes the on-device CNN model upon each new arrival. The complete inference cycle from UDP reception to posture label emission executes within approximately 20 ms, satisfying real-time feedback requirements for postural monitoring applications.

The architectural choice of distributing sensing across two dedicated nodes and consolidating inference on a third reflects a deliberate trade-off. Placing inference on a separate node decouples the 50 Hz acquisition loop from the heavier computation of TFLite Micro inference, avoiding jitter in the sensor sampling cadence. UDP was selected over TCP as the transport protocol for its connectionless, low-overhead nature: occasional packet loss is tolerable given the quasi-static nature of postural signals, whereas the retransmission overhead of TCP would introduce variable latency incompatible with a fixed-rate acquisition loop.

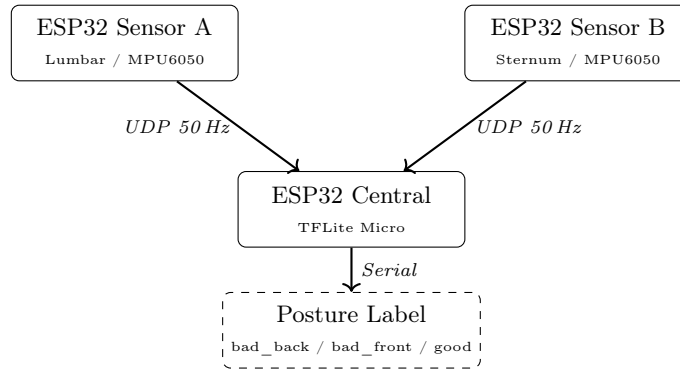


Fig. 1: System architecture. Two sensor nodes stream raw IMU data to the central inference node via UDP. All processing occurs on-device.

2.1 Hardware

The system comprises three ESP32-WROOM-32 microcontrollers and two MPU6050 six-axis IMUs. Two ESP32 nodes act as data acquisition units; a third performs all inference. Table 1 lists the key hardware specifications.

Table 1: Hardware specifications.

Component	Parameter	Value
ESP32-WROOM-32	CPU	Xtensa LX6 dual-core, up to 240 MHz
	SRAM	520 KB
	Flash	4 MB (SPI)
	Wireless	IEEE 802.11 b/g/n (2.4 GHz), Bluetooth 4.2
	GPIO	34 programmable pins
	Supply voltage	3.3 V (on-board regulator from 5 V USB)
MPU6050	Axes	3-axis accelerometer + 3-axis gyroscope
	ADC resolution	16-bit per axis
	Accel range	± 2 / ± 4 / ± 8 / ± 16 g (configurable)
	Gyro range	± 250 / ± 500 / ± 1000 / ± 2000 °/s
	Interface	I ² C (up to 400 kHz)
	Supply voltage	3.3 V, current draw ~ 3.9 mA active
Communication	Protocol	UDP over WiFi 802.11n, port 4210
	Sampling rate	50 Hz per sensor node (20 ms period)
Software	Firmware	Arduino IDE 2.x, TFLite Micro (ESP32 core bundled)
	Training	Python 3, TensorFlow 2.x, scikit-learn, Google Colab

2.2 Sensor Placement

The selection of anatomical placement sites was motivated by the biomechanical requirements of each target class, validated by the exploratory data analysis presented in Section 4.

The **lumbar node** is positioned on the lower back, aligned with the lumbar spine. In this location, the accelerometer axis **ax** carries a stable, high gravitational projection ($\approx 16\,600$ LSB) when the torso is upright, and shifts significantly ($\approx 8\,800$ LSB) when the user reclines. This makes the lumbar node the primary discriminator for **bad_back**: the class is completely isolated in the **ax-az** subspace with no overlap against the other two classes.

The **sternum node** is positioned on the chest. The vertical axis **az** at this location is the only single feature in the entire 12-dimensional space that separates all three classes simultaneously: **bad_back** ($\approx +11\,500$ LSB), **good** ($\approx +2\,600$ LSB), and **bad_front** ($\approx -3\,800$ LSB) produce a monotonic, non-overlapping ordering. This separation is not recoverable from the lumbar node alone, where **bad_front** and **good** are indistinguishable in any 2D feature subspace.

A single-node configuration is therefore insufficient for this three-class problem: the lumbar node trivially identifies backward lean but cannot distinguish forward lean from correct posture; the sternum node provides the complementary

signal necessary to resolve that boundary. The dual-node design is the minimum hardware configuration that makes all three classes linearly separable in the combined 12-dimensional feature space.

2.3 Communication

Each sensor node reads the MPU6050 at 50 Hz via I²C and transmits a comma-separated ASCII datagram to the central node over UDP. The datagram follows a fixed format:

```
NODE_ID,ax,ay,az,gx,gy,gz,t_ms
```

Listing 1.1: UDP packet format (sensor → central).

where `NODE_ID` is the string identifier of the sending node ("lombard" or "esterno"), `ax–gz` are signed 16-bit raw IMU readings, and `t_ms` is the milliseconds elapsed since the node's last boot. The packet fits within 128 bytes; the central allocates a 256-byte receive buffer. The sensor acquisition loop is shown in Listing 1.2.

```
1 void loop() {
2   int16_t ax, ay, az, gx, gy, gz;
3   mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
4
5   char packet[128];
6   snprintf(packet, sizeof(packet),
7            "%s,%d,%d,%d,%d,%d,%d,%lu",
8            NODE_ID, ax, ay, az, gx, gy, gz, millis());
9
10  udp.beginPacket(CENTRAL_IP, UDP_PORT);
11  udp.print(packet);
12  udp.endPacket();
13  delay(20); // 50 Hz
14 }
```

Listing 1.2: Sensor node acquisition and transmission loop (50 Hz).

UDP was selected over TCP for its connectionless, zero-handshake nature. At 50 Hz, the inter-packet interval is 20 ms; occasional packet loss produces a gap in the sliding window but does not stall the acquisition loop, which a TCP retransmission timeout would.

On the central node, each received packet is tokenized with `strtok` on the comma delimiter. Packets with fewer than seven fields are silently discarded. The node identifier routes the six float values into one of two receive buffers (`lom[6]` or `est[6]`). Inference is triggered only when both buffers have been updated since the last inference cycle (the `est_ready && lom_ready` gate), after which both flags are cleared. The 12-element feature row is assembled with the lumbar values in positions 0–5 and the sternum values in positions 6–11, matching the column order used during training:

```
1 float row[12] = {
2   lom[0], lom[1], lom[2], lom[3], lom[4], lom[5], // lombard
3   est[0], est[1], est[2], est[3], est[4], est[5] // esterno
4 };
```

Listing 1.3: Feature row construction on the central node.

The `t_ms` field is not used by the central node at inference time; it was recorded during data collection to enable the nearest-neighbor temporal merge described in Section 5.1. Table 2 summarises the network configuration of all three nodes.

Table 2: Network configuration of the three ESP32 nodes.

Node	Parameter	Value
Sensor A (lumbar)	Placement	Lower back
	<code>NODE_ID</code>	<code>"lombar"</code>
	Destination	Central node, port 4210
	Sampling rate	50 Hz (20 ms period)
Sensor B (sternum)	Placement	Chest (sternum)
	<code>NODE_ID</code>	<code>"esterno"</code>
	Destination	Central node, port 4210
	Sampling rate	50 Hz (20 ms period)
Central (inference)	Role	UDP listener + TFLite Micro inference
	Listening port	4210
	Receive buffer	256 bytes
	Inference gate	<code>est_ready && lom_ready</code>

3 Dataset and Data Collection

3.1 Collection Protocol

The dataset was acquired using the distributed architecture described in Section 2, consisting of two ESP32 sensor nodes equipped with MPU6050 IMUs. One node was positioned on the lumbar region and the other on the sternum. Both nodes sampled accelerometer and gyroscope data at 50,Hz and transmitted measurements independently via UDP over WiFi.

Data collection was performed through controlled recording sessions in which the subject intentionally maintained predefined trunk orientations associated with the target posture classes. Each session lasted approximately 20 seconds. A total of 25 recording sessions were collected, corresponding to 5 sessions for each final posture category (that first was both standing and seated positions).

Table 3: Raw dataset structure before synchronization and merging.

Column	Type	Description
<code>node</code>	string	Identifier of the transmitting sensor node (<code>lombard</code> or <code>esterno</code>)
<code>ax, ay, az</code>	int16	Raw accelerometer readings in MPU6050 LSB units
<code>gx, gy, gz</code>	int16	Raw gyroscope readings in MPU6050 LSB units
<code>t_ms</code>	int64	Milliseconds elapsed since ESP32 boot
<code>label</code>	string	Posture label associated with the recording session
<code>session</code>	int	Unique identifier of the acquisition session

3.2 Class Merging

The original five classes were reduced to three by collapsing the seated/standing distinction (Table 4). The EDA (Sect. 4) confirmed that the sensors measure trunk orientation, which is geometrically identical regardless of whether the user is seated or standing, justifying the merge.

Table 4: Mapping from original acquisition labels to final posture classes.

Final Class	Original Classes	Samples
<code>bad_back</code>	<code>seated_bad_back</code>	9,644
<code>bad_front</code>	<code>standing_bad_front, seated_bad_front</code>	20,425
<code>good</code>	<code>standing_good, seated_good</code>	19,496

4 Exploratory Data Analysis

The objective of the exploratory analysis is threefold: (i) validate dataset integrity, (ii) identify the most discriminative sensor features, and (iii) assess whether the dual-node configuration is justified by the data.

All analyses were performed before normalization and window segmentation, using the raw synchronized sensor streams.

4.1 Data Integrity and Acquisition Consistency

The raw dataset contains 49,565 sensor records with zero missing values and zero duplicated rows. Since each physical posture observation generates one packet from the lumbar node and one from the sternum node, balanced acquisition between both nodes is expected.

The final counts confirm near-perfect synchronization stability: 24,781 samples from the lumbar node and 24,784 from the sternum node. The difference

of only three samples across the entire acquisition process indicates that both ESP32 devices maintained a stable 50 Hz acquisition rate with negligible UDP packet loss.

Additionally, no corrupted packets or malformed sensor rows were detected during CSV parsing. All IMU channels remained within the expected MPU6050 operating range throughout the recordings, indicating stable sensor operation and consistent power delivery during acquisition.

4.2 Class Distribution

Figure 2 presents the distribution of samples after merging the original five posture categories into the three final classes.

`bad_front` and `good` are approximately balanced, with 20,425 and 19,496 samples respectively, while `bad_back` contains 9,644 samples. This imbalance originates directly from the class merging strategy: `bad_back` corresponds to a single original class, whereas the remaining categories each merge two original acquisition classes.

Although moderately imbalanced (approximately 2:1), the distribution remains acceptable for supervised learning. The imbalance is later compensated through class-weighted optimization during CNN training.

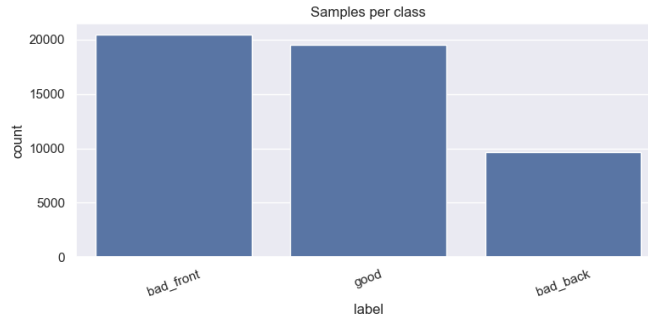


Fig. 2: Distribution of samples across the three final posture classes after class merging.

4.3 Feature Distribution Analysis

To evaluate the discriminative capacity of each sensor channel, violin plots were generated for all accelerometer and gyroscope features of both nodes.

The analysis reveals a clear pattern: accelerometer channels dominate the classification problem, whereas gyroscope signals contribute comparatively little information. This is expected because the task primarily involves quasi-static trunk orientation rather than dynamic motion.

Lumbar Node Figure 3 shows the feature distributions for the lumbar sensor.

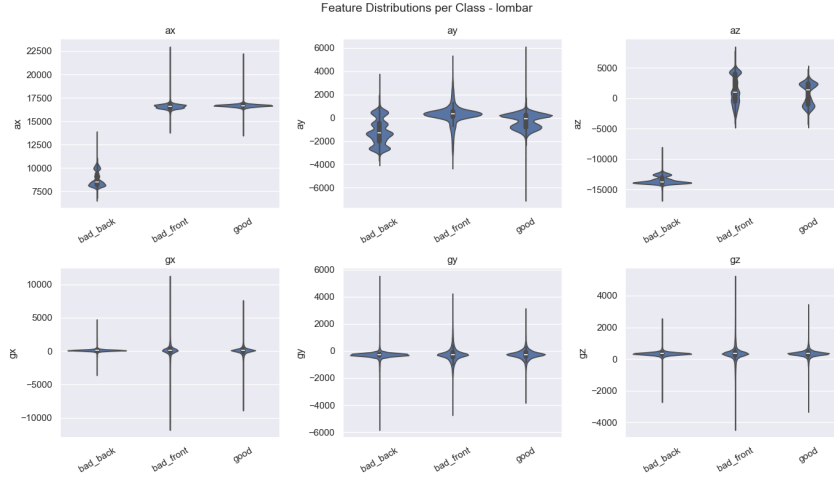


Fig. 3: Feature distributions per class for the lumbar node. Accelerometer channels strongly separate **bad_back** from the remaining classes, while gyroscope channels exhibit large overlap.

The accelerometer axis **ax** is the strongest lumbar feature. **bad_back** forms a compact distribution centred around 8,800 LSB, while both **bad_front** and **good** cluster near 16,600 LSB with minimal overlap. This reflects the large gravitational projection shift produced by backward torso inclination.

The **az** axis reinforces this separation. **bad_back** occupies strongly negative values (approximately $-13,500$ LSB), whereas the remaining two classes remain near zero. However, **bad_front** and **good** still overlap substantially on all lumbar features, indicating that the lumbar node alone cannot fully resolve the three-class problem.

In contrast, gyroscope channels remain centred near zero for all classes and show high variance with extensive overlap. This indicates that angular velocity is not a stable discriminative cue for static posture recognition.

Sternum Node Figure 4 presents the feature distributions for the sternum node.

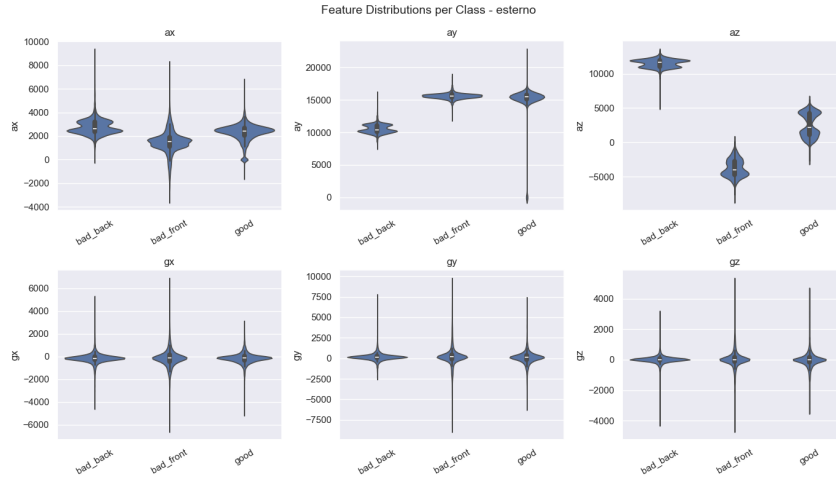


Fig. 4: Feature distributions per class for the sternum node. The accelerometer axis **az** produces near-complete separation between all three posture classes.

The sternum accelerometer axis **az** is the most discriminative feature in the entire dataset. Unlike the lumbar features, it produces a monotonic ordering across all three classes with almost no overlap: **bad_back** occupies strongly positive values, **good** remains near moderate positive values, and **bad_front** shifts into negative values.

This behaviour directly reflects torso pitch orientation and provides the critical information required to separate **bad_front** from **good**, a distinction not recoverable from the lumbar node alone.

The sternum **ay** axis also clearly separates **bad_back** from the remaining classes, although with higher within-class variance for **good**, indicating natural variability in correct posture orientation between sessions.

As with the lumbar node, gyroscope channels show limited discriminative power.

4.4 Pairwise Feature Relationships

To better visualize class separability, Figure 5 shows the most discriminative 2D accelerometer feature spaces for each node.



Fig. 5: Pairwise accelerometer feature distributions. Left: lumbar ax versus az . Right: sternum ay versus az .

The lumbar feature space isolates **bad_back** extremely well, forming a compact cluster with no visible overlap against the other two classes. However, **bad_front** and **good** remain almost completely mixed.

The sternum feature space exhibits substantially better global separability. All three classes occupy distinct regions, with the az axis acting as the dominant separation boundary between **bad_front** and **good**.

These findings validate the dual-node architecture: the lumbar sensor robustly detects backward leaning, while the sternum sensor provides the complementary information necessary to distinguish forward leaning from correct posture.

4.5 Implications for Model Design

The EDA directly influenced multiple architectural decisions in the final system.

First, the strong discriminative structure observed in raw accelerometer signals supports the use of lightweight 1D CNNs operating directly on minimally processed sensor data, avoiding manual feature engineering.

Second, the complementary behaviour of the two sensor nodes validates the decision to use a dual-node configuration rather than a single wearable IMU.

Third, the weak contribution of gyroscope channels suggests that the task is primarily orientation-driven rather than motion-driven, explaining why temporal augmentation strategies such as time shifting later degrade performance.

Finally, the low intra-session noise and strong cluster compactness justify the use of sliding temporal windows, allowing the CNN to learn stable posture patterns from short continuous sequences rather than isolated samples.

4.6 Synchronization Validation

To verify acquisition consistency between nodes, the sample counts of both sensor streams were compared for every recording session. The maximum discrepancy observed was four samples, corresponding to less than 80 ms at 50 Hz.

This confirms that both ESP32 nodes maintained highly stable acquisition rates throughout the data collection process. Consequently, the nearest-neighbor temporal synchronization described in Section 5.1 is expected to produce highly accurate sensor pairings with negligible temporal mismatch. “

5 Preprocessing Pipeline

The preprocessing pipeline transforms two asynchronous IMU streams into a normalized 3D tensor suitable for temporal deep learning:

$$\text{Raw CSV} \xrightarrow{\text{session merge}} \mathbb{R}^{24781 \times 12} \xrightarrow{\text{windowing}} \mathbb{R}^{457 \times 100 \times 12} \xrightarrow{\text{normalization}} X_{\text{norm}}$$

The pipeline is intentionally lightweight and deterministic so that the exact same operations can be replicated during embedded deployment on the ESP32.

5.1 Session-Aware Node Synchronization

The lombar and esterno nodes operate as fully independent ESP32 devices, each maintaining its own local timestamp counter reset at boot time. Consequently, timestamps are only meaningful within the same acquisition session and cannot be aligned globally across recordings.

Synchronization is therefore performed independently for each session using nearest-neighbor temporal matching with `pandas.merge_asof`. For every lombar sample, the temporally closest esterno sample is selected:

```

1 for session in sorted(data["session"].unique()):
2
3     l = lombar[lombar["session"] == session] \
4         .sort_values("t_ms")
5
6     e = esterno[esterno["session"] == session] \
7         .sort_values("t_ms")
8
9     l = l[["t_ms"] + features].rename(
10         columns={f: f"{f}_lmb" for f in features}
11     )
12
13     e = e[["t_ms"] + features].rename(
14         columns={f: f"{f}_est" for f in features}
15     )
16
17     merged = merge_asof(
18         l,
19         e,
20         on="t_ms",
21         direction="nearest"
22     )
23
24     merged["label"] = label
25     merged["session"] = session

```

Listing 1.4: Session-aware nearest-neighbor synchronization.

Session-wise synchronization is critical. A global merge would incorrectly pair samples from unrelated recordings because both devices restart their internal clocks at the beginning of every session.

The final synchronized dataset contains 24,781 aligned samples with 12 sensor features:

6 accelerometer/gyroscope channels (lombar) + 6 channels (esterno)

The synchronization quality was verified during EDA: the maximum inter-node sample count difference was only four samples (<80 ms at 50 Hz), confirming negligible packet loss and near-perfect temporal alignment.

5.2 Sliding Window Segmentation

Deep learning models require fixed-length temporal inputs rather than continuous streams. The merged signal is therefore segmented into overlapping windows using a sliding-window strategy.

Windowing is performed independently for each session so that no sample window crosses recording boundaries. This prevents temporal contamination between sessions and preserves the integrity of grouped cross-validation.

Table 5: Sliding-window configuration and resulting tensor dimensions.

Parameter	Value	Purpose / Effect
WINDOW_SIZE	100 samples	2 s temporal context at 50 Hz
STRIDE	50 samples	50 % overlap between consecutive windows
Tensor shape	(457, 100, 12)	457 windows, 100 timesteps, 12 channels
Class distribution	189 / 179 / 89	bad_front / good / bad_back

The 50 % overlap approximately doubles the number of training samples without requiring additional data collection, while still preserving temporal continuity between adjacent windows.

Each generated window inherits the original session identifier, enabling strict session-aware validation in later stages.

5.3 Feature Standardization

Raw MPU6050 outputs are represented in integer LSB units and exhibit substantially different numerical ranges across channels. For example:

$$\text{ax_lmb} \approx 8,000\text{--}18,000 \quad \text{gz_est} \approx -4,000\text{--}+4,000$$

Without normalization, large-magnitude features would dominate the gradient updates during optimization, biasing the learning process.

Each feature is therefore standardized using z-score normalization:

$$z = \frac{x - \mu}{\sigma}$$

where μ and σ are computed exclusively on the training partition of each fold to prevent information leakage.

The fitted normalization parameters are exported to `deploy_params.json` and embedded directly into the ESP32 firmware, ensuring numerical consistency between offline training and real-time inference.

5.4 Leakage-Free Cross-Validation

Because consecutive windows overlap by 50 %, standard K-Fold validation would place nearly identical temporal segments in both training and testing sets, artificially inflating performance.

To avoid this issue, evaluation uses **Stratified Group K-Fold** cross-validation ($k = 5$), with the recording session acting as the grouping variable.

This strategy guarantees:

- All windows from the same session remain in a single fold.
- Class distribution is approximately preserved across folds.
- Test sessions are completely unseen during training.
- No temporal leakage occurs between train and test partitions.

A verification step confirmed zero shared session identifiers between train and test subsets for every fold.

The resulting evaluation protocol closely matches the intended real-world deployment scenario, where the model must generalize to previously unseen recording sessions.

6 Model Training

6.1 Classical Machine Learning Baselines

To establish non-embedded upper-bound references, two classical machine learning models were trained using handcrafted statistical descriptors computed from each temporal window.

For every one of the 12 sensor channels, four statistical features were extracted:

$$\{\text{mean, std, min, max}\}$$

resulting in a 48-dimensional feature vector per window.

The evaluated models were:

- Random Forest
- Support Vector Machine (SVM)

Table 6: Classical baseline performance using Stratified Group K-Fold ($k = 5$).

Model	Accuracy	Best Parameters	Deployable
Random Forest	1.000 ± 0.000	$n = 171$, depth= 30	No
SVM	1.000 ± 0.000	polynomial, $C = 3.15$	No

Although both models achieve perfect cross-validated accuracy, these results should not be interpreted as evidence of strong generalization. Given the small and homogeneous dataset, the models likely memorize session-specific motion signatures rather than learning robust posture representations transferable to new subjects or environments.

Additionally, neither model is suitable for ESP32 deployment due to memory, latency, and implementation constraints.

6.2 Baseline 1D CNN

Unlike the classical baselines, the CNN operates directly on the raw temporal signal without manual feature engineering. Each input window has shape:

$$100 \times 12$$

corresponding to 100 timesteps and 12 synchronized sensor channels.

The architecture was intentionally designed to remain lightweight enough for microcontroller deployment:

```

1 model = models.Sequential([
2
3     layers.Conv1D(
4         32,
5         kernel_size=5,
6         activation="relu",
7         input_shape=(n_steps, n_features)
8     ),
9
10    layers.Conv1D(
11        64,
12        kernel_size=3,
13        activation="relu"
14    ),
15
16    layers.GlobalAveragePooling1D(),
17
18    layers.Dense(32, activation="relu"),
19
20    layers.Dropout(0.3),
21
22    layers.Dense(n_classes, activation="softmax")
23 ])

```

Listing 1.5: Baseline 1D CNN architecture (v1).

The key architectural decision is the use of `GlobalAveragePooling1D` instead of `Flatten`. This drastically reduces the number of trainable parameters and

lowers SRAM requirements, which is essential for deployment within the ESP32 memory budget (520 KB SRAM).

Training uses:

- Adam optimizer
- Sparse categorical cross-entropy loss
- Early stopping on validation loss (patience = 10)

6.3 Baseline CNN Performance

The baseline CNN achieves:

$$0.751 \pm 0.243$$

cross-validated accuracy.

While the mean accuracy is acceptable, the extremely high standard deviation reveals unstable generalization across folds. Some folds achieve perfect classification, whereas others fall below 0.600.

This variability is not caused by stochastic training noise. Instead, it reflects strong session dependency within the dataset: performance changes substantially depending on which recording sessions are allocated to the test fold.

The baseline CNN therefore learns partially session-specific temporal patterns rather than fully generalized posture representations. This finding motivated the subsequent introduction of:

- data augmentation,
- regularization improvements,
- quantization-aware optimization,
- and architecture refinements for better cross-session robustness.

7 Fine-Tuning

7.1 Gaussian Noise Augmentation

Two copies of each training window are generated with Gaussian noise scaled to 5% of the global signal standard deviation, tripling the effective training set size. Augmentation is applied *inside each fold on the training set only* to prevent data leakage between training and evaluation.

```

1 def augment_noise(X_train, y_train, noise_factor=0.05, n_copies=2):
2     X_aug, y_aug = [X_train], [y_train]
3     for _ in range(n_copies):
4         noise = np.random.normal(0, noise_factor * X_train.std(),
5                                   X_train.shape)
6         X_aug.append(X_train + noise)
7         y_aug.append(y_train)
8     return np.concatenate(X_aug), np.concatenate(y_aug)

```

Listing 1.6: Gaussian noise augmentation (applied per fold, training set only).

Result: 0.908 ± 0.116 . The augmented dataset improves generalization by reducing sensitivity to session-specific signal patterns. Recall on the good class increases from 46.9% to 76.5%.

7.2 Noise + Time Shifting

A temporal augmentation strategy is introduced by applying random time shifts of up to 10 samples, increasing the training set size by a factor of 5.

Result: 0.881 ± 0.123 . Performance decreases relative to noise-only augmentation. For static posture classification, absolute temporal position does not carry discriminative information, and artificial temporal distortion introduces non-physiological patterns that degrade generalization. Gaussian noise is therefore retained as the optimal augmentation strategy.

7.3 Architecture v2: Batch Normalization, L2 Regularization, and Dropout

The baseline CNN architecture is extended with additional regularization mechanisms:

- Batch Normalization after each Conv1D layer to stabilise training dynamics.
- L2 regularization ($\lambda = 0.001$) applied to the dense layer weights.
- Dropout increased from 0.3 to 0.5 to reduce co-adaptation.

Validation leakage issue: an initial evaluation mistakenly used the test fold as the early stopping validation set, resulting in an inflated performance estimate of 0.998 ± 0.005 . This constitutes data leakage and is not a valid result.

The evaluation procedure was corrected by introducing an inner validation split within each training fold, ensuring strict separation between training, validation, and test sets:

```

1 train_idx_inner, val_idx_inner = train_test_split(
2     train_idx,
3     test_size=0.2,
4     stratify=y_enc[train_idx],
5     random_state=42
6 )
7
8 # Early stopping monitors only validation subset
9 model.fit(
10     X_train_aug, y_train_aug,
11     validation_data=(X_val, y_val),
12     ...
13 )
14
15 acc = model.evaluate(X_test, y_test) # fully held-out evaluation

```

Listing 1.7: Corrected inner validation split within each fold.

Final corrected result: 0.943 ± 0.114 . This confirms that the previously reported performance was inflated due to leakage. The corrected architecture still improves significantly over the baseline (+19.2 pp) and represents the final deployable model.

7.4 Comparison of All Evaluated Models

Table 7: Comparison of evaluated models using Stratified Group K-Fold cross-validation ($k = 5$).

Model	Accuracy Std		Status
CNN v1 (baseline)	0.751	0.243	Reference baseline
CNN v1 + Gaussian Noise	0.908	0.116	Best-performing augmentation strategy
CNN v1 + Noise + Time Shift	0.881	0.123	Temporal augmentation degrades performance
CNN v2 (invalid evaluation)	0.998	0.005	Data leakage (validation on test set)
CNN v2 (final)	0.943	0.114	Deployable model

8 TFLite Export and Deployment

8.1 Final Model Training and Int8 Quantization

The final model is retrained using the full available training set (457 windows, augmented to 1,371 samples). No validation split is used at this stage, as the model is already selected and fixed after cross-validation.

The model is then converted to TensorFlow Lite using full integer quantization:

```

1 converter = tf.lite.TFLiteConverter.from_keras_model(final_model)
2 converter.optimizations = [tf.lite.Optimize.DEFAULT]
3
4 converter.representative_dataset = representative_dataset # calibration
   subset
5 converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
6
7 converter.inference_input_type = tf.int8
8 converter.inference_output_type = tf.int8
9
10 tflite_model = converter.convert()
```

Listing 1.8: TFLite int8 quantization pipeline.

Int8 quantization maps 32-bit floating point weights and activations to 8-bit integers, reducing model size by approximately $4\times$ and enabling efficient inference on resource-constrained embedded hardware.

8.2 Quantization Parameters

Table 8: TFLite quantization parameters extracted from the deployed model.

Parameter	Value	Role
INPUT_SCALE	0.09173505753	Input dequantization scale
INPUT_ZERO_POINT	-3	Input quantization offset
OUTPUT_SCALE	0.00390625	Output probability scaling factor
OUTPUT_ZERO_POINT	-128	Output quantization offset

8.3 Quantization and Dequantization Mapping

The quantization mapping applied in firmware is defined as:

$$q = \text{clip}\left(\left\lfloor \frac{z}{\text{INPUT_SCALE}} \right\rfloor + \text{INPUT_ZERO_POINT}, -128, 127\right) \quad (1)$$

$$p_i = (\text{output}[i] - \text{OUTPUT_ZERO_POINT}) \cdot \text{OUTPUT_SCALE} \quad (2)$$

where z denotes the StandardScaler-normalised sensor input and p_i represents the dequantized class probability.

8.4 Embedding the Model in Flash (model.h)

The ESP32 environment does not include a runtime filesystem for model storage. Therefore, the compiled `.tflite` model is embedded directly into flash as a static C array.

```

1 with open("model.tflite", "rb") as f:
2     data = f.read()
3
4 with open("model.h", "w") as f:
5     f.write("#ifndef MODEL_H\n#define MODEL_H\n#define MODEL_H\n\n")
6     f.write(f"const unsigned int model_tflite_len = {len(data)};\n")
7     f.write("alignas(8) const unsigned char model_tflite[] = {\n ")
8     f.write(", ".join(f"0x{b:02x}" for b in data))
9     f.write("\n};\n\n#endif\n")

```

Listing 1.9: Conversion of TFLite binary to C header.

The resulting array is placed in the `.rodata` section and stored in flash memory. At runtime, TensorFlow Lite Micro accesses the model directly from this region, while activations are allocated in the dedicated tensor arena.

This ensures that the model weights remain immutable and are not dynamically copied into heap memory during inference.

9 Firmware Architecture

9.1 TFLite Micro Components

Tensor Arena The `tensor_arena` is a statically allocated memory region used by TensorFlow Lite Micro to store all intermediate tensors, including activations, input, and output buffers. No dynamic memory allocation is performed at runtime.

```
1 constexpr int kArenaSize = 40 * 1024;    // 40 KB
2 uint8_t tensor_arena[kArenaSize];        // static allocation in RAM
```

Listing 1.10: Static tensor arena allocation.

Memory usage after allocation can be inspected as follows:

```
1 Serial.printf("Arena used: %u bytes\n",
2             interpreter->arena_used_bytes());
```

MicroMutableOpResolver The `MicroMutableOpResolver` registers only the operators required by the model, reducing flash footprint and binary size. Table 9 summarises the deployed operator set.

Table 9: TensorFlow Lite Micro operators used in the deployed model.

Operator	TFLite Op	Role in Model
Conv1D	CONV_2D	Feature extraction layers
FullyConnected	FULLY_CONNECTED	Classification head
GlobalAveragePooling	MEAN	Temporal aggregation
Softmax	SOFTMAX	Output probability layer
Reshape	RESHAPE	Tensor dimension alignment
Quantization	QUANTIZE	Input int8 mapping
Dequantization	DEQUANTIZE	Output scaling
ExpandDims	EXPAND_DIMS	Batch dimension handling
Element-wise Mul	MUL	Fused scaling operations
Element-wise Add	ADD	Bias and normalization offsets

Input and Output Tensors After `AllocateTensors()`, fixed pointers to input and output buffers are stored and reused for each inference cycle:

```
1 input  = interpreter->input(0);    // [1, 100, 12], int8
2 output = interpreter->output(0);   // [1, 3], int8
```

9.2 Inference Pipeline

For each incoming synchronized sample pair (lombar + esterno), the system executes a streaming inference pipeline:

1. **Feature fusion:** concatenate 6 accelerometer/gyroscope values from each node into a 12-dimensional feature vector.
2. **Sliding window update:** shift the existing buffer by one timestep and append the new observation at the last position.
3. **Inference trigger:** execute the model only when the buffer reaches 100 timesteps.
4. **Normalization and quantization:** apply StandardScaler transformation followed by int8 quantization (Eq. 1).
5. **Model execution:** run `Invoke()` on the TensorFlow Lite Micro interpreter.
6. **Post-processing:** dequantize output logits and apply argmax decision rule (Eq. 2).

```

1  int idx = 0;
2
3  // Quantization + input tensor filling
4  for (int t = 0; t < N_STEPS; t++) {
5      for (int f = 0; f < N_FEATURES; f++) {
6
7          float z = (window[t][f] - MEAN[f]) / STD[f];
8
9          int q = (int)round(z / INPUT_SCALE) + INPUT_ZP;
10
11         if (q < -128) q = -128;
12         if (q > 127) q = 127;
13
14         input->data.int8[idx++] = (int8_t)q;
15     }
16 }
17
18 // Inference
19 interpreter->Invoke();
20
21 // Post-processing
22 int best = 0;
23 float bestp = -1;
24
25 for (int i = 0; i < N_OUTPUTS; i++) {
26     float p = (output->data.int8[i] - OUTPUT_ZP) * OUTPUT_SCALE;
27
28     if (p > bestp) {
29         bestp = p;
30         best = i;
31     }
32 }
33
34 Serial.printf("Posture: %s (%.2f)\n", LABELS[best], bestp);

```

Listing 1.11: Real-time inference loop on ESP32 central node.

Latency: After an initial warm-up period of 2 seconds (window filling), the system performs continuous inference with an average latency of approximately 20 ms per evaluation cycle.

The sliding-window architecture ensures uninterrupted real-time classification without sampling gaps.

10 Discussion and Limitations

10.1 Performance Analysis

The final model achieves a mean accuracy of 94.3% under Stratified Group K-Fold cross-validation, reducing the gap between the baseline CNN (75.1%) and the upper bound defined by classical models (100%). The remaining variance ($\pm 11.4\%$) is primarily driven by a single low-performing fold (71.6%), indicating that performance variability is dominated by session-level distribution shifts rather than model capacity.

The most challenging decision boundary is between `bad_front` and `good`. While the esterno `az` axis provides strong discriminative information, intra-class variability in `good` posture (particularly in `ay`) reduces separability in feature space, resulting in a softer decision boundary.

10.2 Limitations

1. **Dataset size and diversity:** The dataset consists of 457 windows derived from a limited number of subjects under controlled conditions. This introduces potential subject-dependent bias and limits generalisation to unseen users.
2. **Synchronization strategy:** The system performs nearest-sample fusion between sensor nodes without explicit timestamp alignment. A temporal mismatch of approximately 10–20 ms is acceptable for static posture classification but would be insufficient for high-frequency motion analysis.
3. **Network configuration dependency:** Sensor nodes rely on a hardcoded central IP address. DHCP reassignment after reboot may lead to communication failure, requiring static IP configuration or mDNS-based discovery.
4. **Output interface limitation:** The current implementation provides only serial output, with no user-facing feedback mechanism for real-time deployment scenarios.

10.3 Future Work

- Extend data collection to additional subjects and environmental conditions to improve robustness against inter-subject variability.
- Introduce explicit timestamp-based validation, rejecting samples where $|t_{\text{lmb}} - t_{\text{est}}| > 25$ ms.
- Integrate real-time feedback mechanisms such as LED indicators or OLED display modules for user-facing deployment.
- Evaluate the finetuned model (`model_finetuned.h`) under real-world conditions and compare error distributions against cross-validation results.
- Implement temporal persistence logic, triggering alerts only when incorrect posture is detected continuously over a defined time window.

11 Conclusion

This work presents a complete embedded posture detection system spanning raw IMU acquisition, signal processing, model training, and on-device inference using TensorFlow Lite Micro on ESP32 hardware.

The system achieves a cross-validated accuracy of 94.3% using a compact 18.9KB int8-quantized convolutional neural network, with an inference latency of approximately 20 ms, fully executed on-device.

Key contributions include a Stratified Group K-Fold evaluation strategy that prevents window-level leakage across sessions, the identification and correction of a validation leakage issue caused by improper use of the test set for early stopping, and an empirical analysis of augmentation strategies showing that Gaussian noise improves both mean performance (+15.7 pp) and stability (reducing standard deviation from 0.243 to 0.116), whereas temporal shifting degrades performance in static posture recognition tasks.

Overall, the results demonstrate that sub-20 KB quantized models can achieve practical accuracy on resource-constrained microcontrollers, with a fully reproducible end-to-end embedded machine learning pipeline.

References

1. Ali, M., Siddiq, M.A.B., Pranto, N.K., et al.: Prevalence and predictors of musculoskeletal health complaints among sedentary, monotonous urban workers: A survey in Bangladesh. *PLOS ONE* **18**(4), e0282922 (2023). <https://doi.org/10.1371/journal.pone.0282922>
2. Baloch, Z., et al.: CNN-LSTM-based late sensor fusion for human activity recognition in big data networks. *Wireless Communications and Mobile Computing* **2022**, 3434100 (2022). <https://doi.org/10.1155/2022/3434100>
3. Diab, M.S., Rodriguez-Villegas, E.: Embedded machine learning using microcontrollers in wearable and ambulatory systems for health and care applications: A review. vol. 10, pp. 98450–98474 (2022). <https://doi.org/10.1109/ACCESS.2022.3206782>
4. Donisi, L., Cesarelli, G., Pisani, N., Ponsiglione, A.M., Ricciardi, C., Capodaglio, E.: Wearable sensors and artificial intelligence for physical ergonomics: A systematic review of literature. *Diagnostics* **12**(12), 3048 (2022). <https://doi.org/10.3390/diagnostics12123048>
5. Ferreira, Á., Cunha, J., Tavares, J.M.R.S.: PostureMonitor: Real-Time IMU wearable technology to foster poise and health. In: *Distributed Computing and Artificial Intelligence. Advances in Intelligent Systems and Computing*, vol. 373, pp. 423–430. Springer (2015). https://doi.org/10.1007/978-3-319-19638-1_50
6. Grabara, M.: Prevalence of musculoskeletal disorders among Polish white-collar workers: the role of physical activity and risk factors. *Frontiers in Public Health* **13** (2025). <https://doi.org/10.3389/fpubh.2025.1551728>, 12-month prevalence: lower back pain 65.9%, neck pain 59.8%
7. Jayasinghe, U., Janko, B., Hwang, F., Harwin, W.S.: Classification of static postures with wearable sensors mounted on loose clothing. *Scientific Reports* **13**, 131 (2023). <https://doi.org/10.1038/s41598-022-27306-4>

8. Kim, K.t., Kwon, J.w., Kim, H.s., Ha, S.i., Park, S.k., Jeong, S.: Fast wearable sensor-based foot-ground contact phase classification using a convolutional neural network with sliding-window label overlapping. *Sensors* **20**(17), 4996 (2020). <https://doi.org/10.3390/s20174996>
9. Leone, A., Rescio, G., Caroppo, A., Siciliano, P., Manni, A.: Human postures recognition by accelerometer sensor and ML architecture integrated in embedded platforms: Benchmarking and performance evaluation. *Sensors* **23**(2), 1039 (2023). <https://doi.org/10.3390/s23021039>
10. National Safety Council: Musculoskeletal injuries and illnesses — Injury Facts. <https://injuryfacts.nsc.org/work/safety-topics/musculoskeletal-injuries/> (2024), accessed: 2025